



INFORME DE PROYECTO FINAL

SISTEMA DE VENTA DE PASAJES Tbus

NOMBRE DEL CURSO: TÉCNICAS DE PROGRAMACIÓN
ORIENTADA A OBJETOS

CÓDIGO DEL CURSO: 5076

NOMBRE DEL PROYECTO: Sistema de venta de Pasajes

INTEGRANTES DEL EQUIPO: CELIA CASAS QUINTANA

DOCENTE: MARTIN EDUARDO TORRES RODRIGUEZ

CICLO ACADÉMICO: 2025-5

FECHA DE ENTREGA: 21/09/2025

1. INTRODUCCIÓN

1.1 PRESENTACIÓN DEL PROYECTO

El sistema "TBus - Registro de Venta de Pasajes" es una aplicación de escritorio desarrollada en Java Swing con la finalidad registrar ventas de pasajes de forma eficaz, óptima y sencilla. Está dirigido al uso interno en boleterías o para prácticas académicas, enfocado en la captura de datos del asiento, pasajero, destino y precio. Es de interfaz simple para el personal de ventas y un registro inmediato.

1.2 JUSTIFICACIÓN

En muchas empresas de transporte, la gestión de pasajes se hace de manera manual o con sistemas no bien adaptados a las necesidades del lugar. Provocando desorganización, pérdida de datos y demora en la atención.

Con este proyecto, se busca crear una herramienta informática sencilla y económica, útil en entornos reales. Se eligió Java Swing porque es multiplataforma y facilita la creación de interfaces menos complejas.

1.3 OBJETIVOS GENERALES Y ESPECÍFICOS

OBJETIVO GENERAL

El propósito del proyecto es desarrollar un sistema en Java para automatizar la venta de pasajes para transporte terrestre, optimizando el proceso de venta. Con esta herramienta, se agiliza la atención al cliente, mayor control administrativo de las ventas y la asignación de asientos.

Ante la necesidad de modernización viendo procesos que, en muchos casos, todavía se hacen a mano o con sistemas poco integrados esto genera retrasos, errores en la asignación de asientos, duplicación de datos y una experiencia insatisfactoria para los usuarios.

OBJETIVOS ESPECÍFICOS

- Automatización de la venta de pasajes de una empresa de transporte terrestre
- Definir funciones para el sistema con sus requisitos a cumplir, y responda a las necesidades de la empresa y de los clientes.
- Implementar un registro de pasajeros y clientes que valide sus datos personales, asegurando que la información sea precisa y confiable.
- Gestionar en un módulo las rutas y horarios, donde se puedan registrar viajes con detalles como destino y número de asientos disponibles, y que permita una atención eficaz.
- Diseñar un sistema para asignar asientos en tiempo real, usando colecciones de Java como ArrayList y HashMap, para reflejar la ocupación y evitar la sobreventa.
- Incorporar controles y así validar la información ingresada, manejando errores, evitando datos incompletos o incorrectos.
- Aplicar principios de programación orientada a objetos para que el sistema sea modular, escalable y fácil de mantener.

1.4 ALCANCE DEL SISTEMA

ALCANCE FUNCIONAL

- **Registro de clientes y pasajeros:** Almacena información básica de los clientes.
- **Asignación de asientos:** controlar la disponibilidad de asientos, mostrando al usuario cuáles están libres y cuáles ocupados
- **Venta de pasajes:** Permite la elección del destino de viaje hasta la confirmación de la compra, generando comprobantes de forma automática.
- **Consultas rápidas y reportes:** listados de ventas por fecha, por pasajero o por ruta.
- **Validaciones y manejo de excepciones:** el sistema avisará de los errores (datos mal ingresados, sobreventa de asientos).

ALCANCE TÉCNICO

- **Compatibilidad:** el sistema será ejecutable en equipos con Windows 10 o superior, con un entorno de Java actualizado (JDK 21+).
- **Persistencia temporal:** almacenamiento en colecciones y archivos locales, con posibilidad de migración a una base de datos en etapas posteriores.
- **Escalabilidad:** principios POO para facilitar la integración con nuevas funcionalidades.
- **Lenguaje de programación:** Java (Swing para la interfaz).
- Estructuras de datos: colecciones (ArrayList, HashMap, DefaultTableModel) administrarán la información.

ALCANCE DE USUARIOS

- **Personal administrativo:** El encargado de la atención al cliente, registrará a los pasajeros, procesará ventas el sistema y emitirá reportes.
- **Clientes o pasajeros:** Los clientes experimentarán con el sistema por intermedio del personal que seleccionará los viajes y boletos a adquirir (en el futuro se puede habilitar una interfaz directa al cliente en fases posteriores en la web de la empresa).

LIMITACIONES DEL ALCANCE

- El sistema no incluye conexión a internet para reservas en línea.
- No se cuenta con impresión física de boletos pero puede añadirse en futuras versiones.
- La persistencia es manejada mediante archivos locales y colecciones en memoria, en el futuro con bases de datos robustas (MySQL, PostgreSQL)

2. PLANTEAMIENTO DEL PROBLEMA

DIAGRAMA DE ISHIKAWA



2.1 DESCRIPCIÓN DEL PROBLEMA IDENTIFICADO

En muchas pequeñas empresas de transporte, el proceso de registro es manual o con herramientas improvisadas (papel, hojas de cálculo, aplicaciones muy básicas). El prototipo actual (VentaPasajesGUI en Java Swing) permite registrar ventas en pantalla, sin embargo presenta limitaciones :

- Sin persistencia: los registros quedan sólo en memoria (tabla) pérdida de la información al cerrar el programa.
- Control de asientos insuficiente: no hay verificación robusta de asientos ocupados, lo que facilita duplicidad.
- Sin reportes ni trazabilidad: no hay forma de generar totales diarios, exportar ventas y transacciones.
- Interfaz y usabilidad limitadas: la GUI básica dificulta operaciones masivas.

2.2 RESTRICCIONES Y LIMITACIONES REALISTAS

Técnicas

- El sistema funcionará en equipos con JDK 21+ (en ausencia de Internet).
- Compatible Windows 10+ (principalmente) y Linux en entornos de boletería.

Recursos

- Presupuesto limitado: soluciones con licencias comerciales o hosting puede significar costo adicional a la empresa.

Operacionales

- Necesidad de operar offline (algunas boleterías no tienen conexión estable).
- Bajo nivel de formación técnica de operadores (interfaz simple requerida).

Legales / fiscales

- La facturación, debe cumplir requisitos locales (boletas, comprobantes electrónicas).

Organizacionales

- Resistencia al cambio: el personal puede preferir lo conocido (papel/Excel).

2.3 ALTERNATIVAS DE SOLUCIÓN EVALUADAS

Mejorar la aplicación actual en Java Swing (recomendado como fase 1)

Implica

- Añadir modelos de dominio (Venta), capa VentaService y VentaDAO.
- Persistencia local con SQLite (archivo local).
- Exportar CSV / generar resumen diario.

Pros

- Costo bajo
- No requiere hosting, internet.

Contras

- Interfaz Swing limitada en estética/UX.
- Escalabilidad limitada para el acceso simultáneo desde múltiples puntos.

Migrar UI a JavaFX (fase 2 posible)

Implica

- Reescribir ,adaptarlo a JavaFX (mejor estética, controles modernos).
- Mantener capas Servicio/DAO para reuso.

Pros

- Mejor experiencia de usuario y facilidad para enriquecer UI.
- Ejecución offline.

Contras

- Mayor curva de aprendizaje; esfuerzo de reescritura UI.

Usar solución SaaS de ticketing (comprar/contratar)

Implica

- Contratar un proveedor que se encargue de la venta de pasajes, gestión y reportes.

Pros

- Eficaz implementación (pagos, facturación).
- Soporte y actualizaciones proporcionadas por el proveedor.

Contras

- Dependencia del proveedor; poca personalización.

3. ANTECEDENTES

3.1 Revisión de literatura o proyectos similares

PROPUESTA DE MEJORA EN EL ÁREA DE VENTAS BOLETAJE NACIONAL, AGENCIA DE VIAJES, AC TOURS 2018

Desde la perspectiva del análisis metodológico, y en consonancia con Hernández, Fernández y Baptista (2014), esta investigación adopta un enfoque cuantitativo, caracterizado por ser "un conjunto de procesos, secuencial y probatorio" (p. 4). El diseño es no experimental de corte transeccional, ya que los datos se recogen en su ambiente natural y en un momento específico. Complementariamente, desde el enfoque temporal propuesto por Hurtado (2000), la investigación es de tipo proyectiva, al tomar como modelo una propuesta de solución a un problema práctico en un área específica del conocimiento, partiendo de un diagnóstico preciso de las necesidades actuales con proyección futura. El proceso de diagnóstico mediante la matriz FODA permitió establecer estrategias específicas para maximizar fortalezas y oportunidades, minimizar debilidades y amenazas, así como para aprovechar las capacidades internas de la organización frente al entorno externo.

Desarrollo de un sistema web para dar soporte en la venta de pasajes en la Empresa de Transportes Santacruz Juliaca 2021

El estudio de Yanapa Quispe (2022) desarrolló un sistema web multiusuario con software libre que centralizó la venta de pasajes en Transportes Santacruz, mejorando la eficiencia operativa en 40%, reduciendo costos en 60% mediante soluciones open source, e implementando estrategias de fidelización mediante base de datos centralizada, demostrando la viabilidad técnica de plataformas web-based para empresas de transporte (pp. 21-26).

3.2 REFERENCIAS EN FORMATO APA

Elias Campos, C. P. (2017). *Propuesta de mejora en el área de ventas boletaje nacional, agencia de viajes, AC Tours 2018* (Trabajo de suficiencia profesional) [Tesis de licenciatura, Universidad San Ignacio de Loyola]. Repositorio USIL.

<https://repositorio.usil.edu.pe/server/api/core/bitstreams/037b468c-f736-4ec4-ad30-b36e7eeead06/content> (pp. 33-35)

Yanapa Quispe, E. (2022). *Desarrollo de un sistema web para dar soporte en la venta de pasajes en la Empresa de Transportes Santacruz Juliaca 2021* [Tesis de grado, Universidad Andina Néstor Cáceres Velásquez]. Repositorio Institucional UANCV.

<https://repositorio.uancv.edu.pe/handle/UANCV/2047> (pp. 21-26)

4. REQUERIMIENTOS DEL SISTEMA

4.1 REQUERIMIENTOS FUNCIONALES:

ID	Requerimiento Funcional (RF)	Estado
RF-01	El sistema debe registrar el nombre del pasajero.	Completado
RF-02	El sistema debe validar que el campo nombre no esté vacío.	Completado
RF-03	El sistema debe mostrar una lista de destinos.	Completado
RF-04	El sistema debe permitir seleccionar un destino de la lista.	Completado
RF-05	El sistema debe permitir ingresar un número de asiento.	Completado
RF-06	El sistema debe validar que el campo asiento no esté vacío.	Pendiente
RF-07	El sistema debe validar que no se repitan los números de asiento.	Pendiente
RF-08	El sistema debe permitir ingresar el precio del pasaje.	Completado
RF-09	El sistema debe validar que el campo precio no esté vacío.	Completado
RF-10	El sistema debe validar que el precio sea mayor a 0.	Completado
RF-11	El sistema debe registrar cada venta en una tabla.	Completado
RF-12	El sistema debe permitir registrar múltiples ventas consecutivas.	Completado

RF-13	El sistema debe limpiar los campos después de cada registro.	Completado
RF-14	El sistema debe mostrar un mensaje de confirmación de venta exitosa.	Completado
RF-15	El sistema debe mostrar mensajes claros en caso de error.	Completado
RF-16	El sistema debe permitir desplazarse en la tabla de ventas.	Completado
RF-17	El sistema debe mostrar un encabezado de columnas en la tabla.	Completado
RF-18	El sistema debe mantener los registros en memoria mientras esté abierto.	Completado
RF-19	El sistema debe permitir registrar ventas de destinos añadidos previamente.	Completado
RF-20	El sistema debe permitir consultar las ventas registradas.	Completado
RF-21	El sistema debe permitir eliminar un registro seleccionado.	Pendiente
RF-22	El sistema debe permitir modificar los datos de un registro.	Completado
RF-23	El sistema debe validar entradas no numéricas en asiento.	Pendiente
RF-24	El sistema debe validar entradas no numéricas en precio.	Pendiente
RF-25	El sistema debe calcular el total de ventas realizadas.	Completado

RF-26	El sistema debe mostrar el número total de pasajes vendidos.	Completado
RF-27	El sistema debe permitir exportar la tabla a un archivo CSV.	Pendiente
RF-28	El sistema debe permitir buscar ventas por nombre de pasajero.	Completado
RF-29	El sistema debe permitir buscar ventas por destino.	Completado
RF-30	El sistema debe mostrar el asiento como único por pasajero.	Completado
RF-31	El sistema debe mostrar el precio en formato monetario.	Completado
RF-32	El sistema debe permitir ordenar registros por precio.	Pendiente
RF-33	El sistema debe permitir ordenar registros por nombre.	Completado
RF-34	El sistema debe generar un resumen de ventas diarias.	Completado
RF-35	El sistema debe diferenciar entre ventas registradas y anuladas.	Pendiente
RF-36	El sistema debe permitir guardar temporalmente la última venta.	Completado
RF-37	El sistema debe permitir restaurar un registro eliminado por error.	Pendiente
RF-38	El sistema debe mostrar la fecha y hora de cada registro.	Pendiente

RF-39	El sistema debe permitir cerrar sesión o salir del programa de forma segura.	Pendiente
RF-40	El sistema debe mostrar un mensaje de despedida al cerrar la aplicación.	Pendiente

4.2 REQUERIMIENTOS NO FUNCIONALES Y ACEPTACIÓN

ID	Requerimiento No Funcional (RNF)	Criterio de Aceptación
RNF-01	El sistema debe ser de interfaz intuitiva.	La interfaz debe contener menús y botones claros.
RNF-02	El sistema debe ejecutarse en computadoras con Windows 10 o superior	Compatible con JVM estándar.
RNF-03	El sistema debe estar implementado en Java con Swing	Ejecutable en cualquier máquina con JDK instalado.
RNF-04	El sistema debe consumir menos de 200 MB de memoria RAM.	Validación en ejecución normal.
RNF-05	El sistema debe garantizar integridad en los datos ingresados.	Validación estricta de entradas inválidas.
RNF-06	Mostrar los mensajes en idioma español.	Todo el feedback mostrado en español.
RNF-07	El sistema debe soportar mínimo 500 registros por sesión.	La tabla no debe fallar con grandes volúmenes.

5. HISTORIAS DE USUARIO

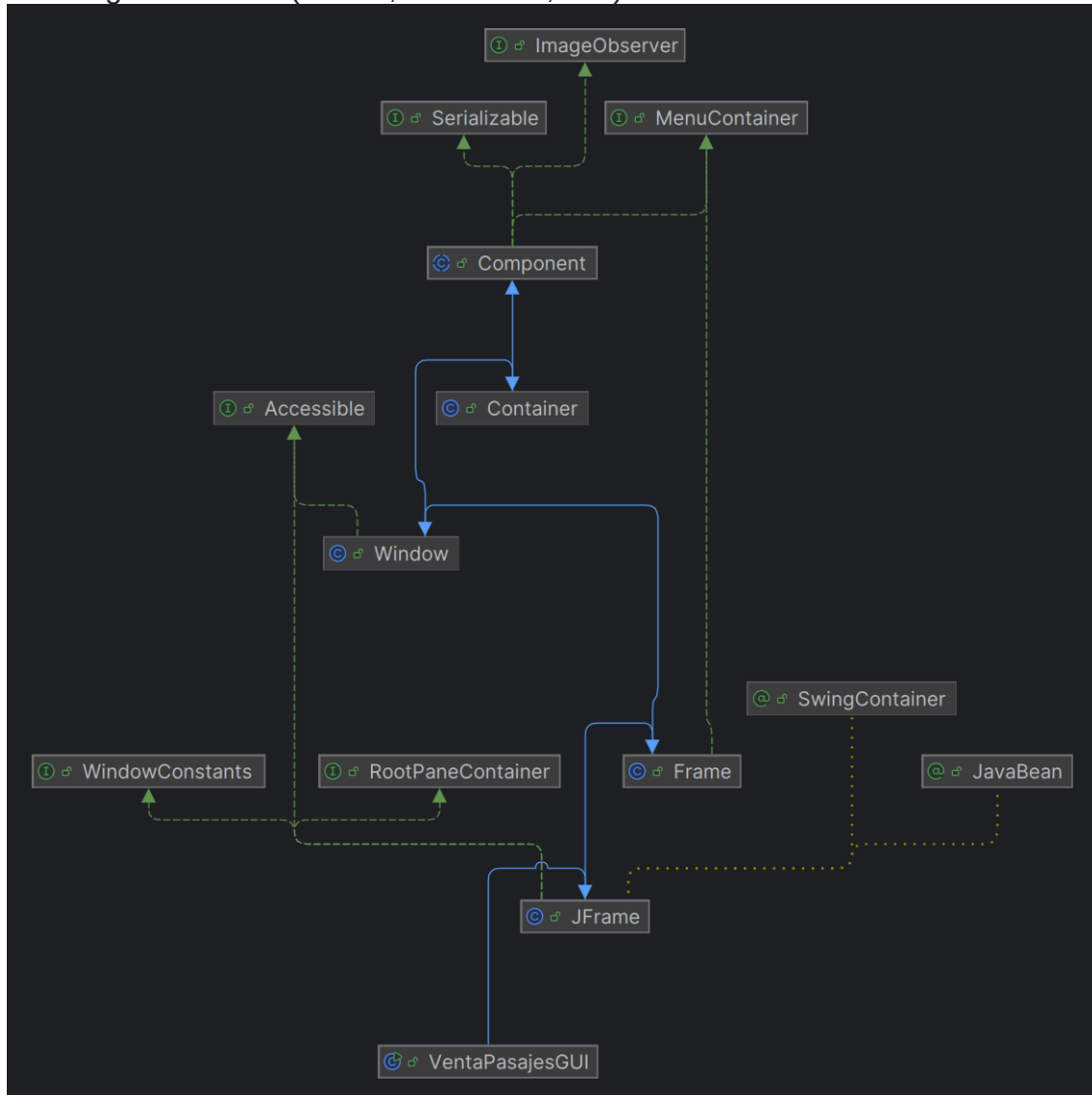
5.1 Tabla con historias de usuario priorizadas y aceptación esperada

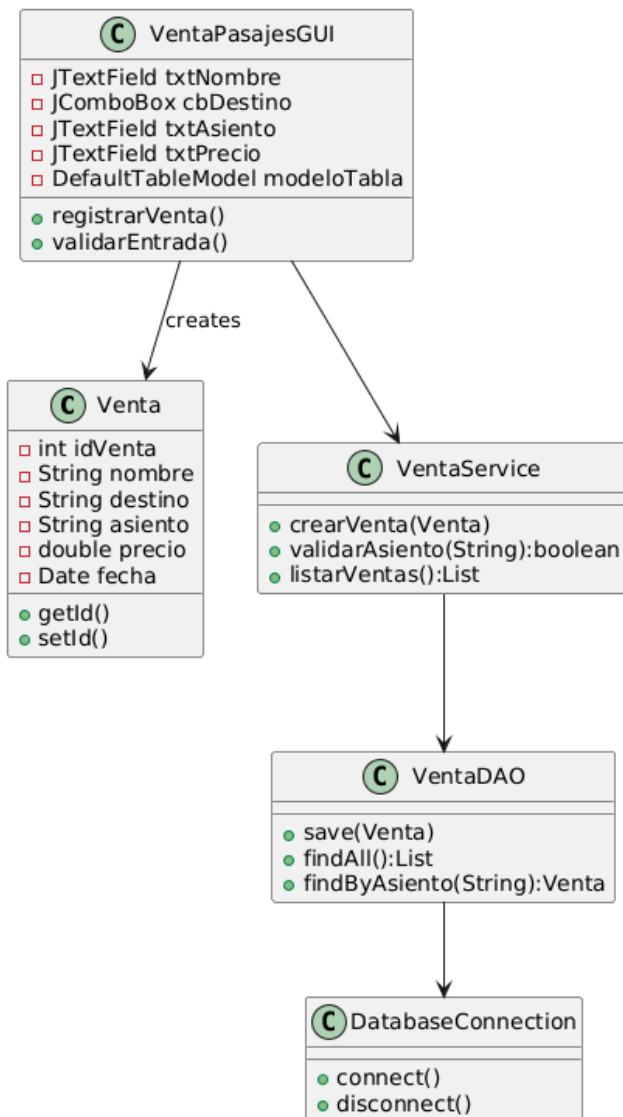
ID	Historia de Usuario	Prioridad	Aceptación Esperada
----	---------------------	-----------	---------------------

HU-01	El vendedor, quiere registrar el nombre del pasajero y asignarle un boleto.	Alta	El sistema no debe permitir campos vacíos en "Nombre".
HU-02	El pasajero, quiere elegir un destino de viaje y asegurar que el pasaje corresponda a su ruta.	Alta	El sistema debe mostrar una lista de destinos predefinidos.
HU-03	El vendedor, asigna un número de asiento para evitar duplicados.	Alta	El sistema debe validar que el asiento no haya sido vendido.
HU-04	El administrador, quiere registrar el precio de cada pasaje para llevar control de ingresos.	Alta	En el sistema se debe ingresar un valor numérico en el campo "Precio".
HU-05	El vendedor requiere visualizar las ventas en una tabla para consultar.	Alta	Los datos deben mostrarse en una tabla con columnas: Nombre, Destino, Asiento, Precio.
HU-06	El vendedor, quiere que se validen los campos vacíos para evitar registros incompletos.	Alta	Mostrar un mensaje de error, si hay campos vacíos.
HU-07	El administrador, requiere que se limpien los campos, luego de finalizar un registro.	Media	Al terminar de registrar, los campos deben reiniciarse automáticamente.
HU-08	El pasajero, requiere que el sistema me muestre mensajes claros si cometo errores al ingresar información.	Media	Los mensajes de error deben ser descriptivos.
HU-09	El administrador, necesita que los datos queden almacenados en memoria mientras la aplicación esté abierta.	Media	El sistema almacena y muestra los registros en la tabla hasta cerrar el programa

6. DISEÑO DEL SISTEMA

6.1 Diagramas UML (clases, relaciones, etc.)





6.2 ARQUITECTURA PROPUESTA

Se propone una arquitectura en capas (layered architecture) facilitará mantenimiento y desarrollo:

- La capa de presentación, desarrollada con Java Swing, gestiona la interfaz de usuario y se encarga de la validación.
- La capa de negocio contiene las reglas empresariales, como la validación de asientos, cálculos y políticas de precios
- La capa de persistencia, mediante el patrón DAO, se encarga de almacenar y recuperar objetos de venta, con implementaciones que pueden variar entre bases de datos en memoria, SQLite o MySQL
- La capa de utilidades/infraestructura proporciona servicios auxiliares como conexión a bases de datos, registro de eventos y exportación de datos en formatos CSV y PDF.
- El despliegue inicial contempla una aplicación de escritorio con conexión opcional a bases de datos locales o remotas, mientras que la arquitectura permite una futura migración a una aplicación web (utilizando frameworks como React o Angular con Spring Boot) sin modificar la lógica del negocio.

6.3 ALCANCE DE LA SOLUCIÓN

- Diseño:
 - La arquitectura del sistema incorpora los modelos de datos esenciales (Venta y Destino)
 - servicios de validación de entradas y prevención de duplicidad de asientos, un módulo de persistencia con implementación inicial en memoria mediante.
- Características Excluidas en Fase Inicial:
 - El diseño actual no incluye -pero contempla para futuras iteraciones- sistemas de autenticación y autorización de usuarios
 - Integración con pasarelas de pago.
 - Facturación electrónica y emisión de boletas.
- Requisitos Técnicos:
 - Compatibilidad con Java 21+.
 - Repositorio Git con control de versiones.
 - Tests unitarios en la capa de servicio (JUnit) focalizadas en las reglas del negocio.

7. IMPLEMENTACIÓN

7.1 Descripción del desarrollo en Java

El tema de la programación orientada a objetos en Java para la construcción de aplicaciones con interfaces gráficas intuitivas y escalables. específicamente la biblioteca Swing, ha sido una gran herramienta para el desarrollo de interfaces gráficas, debido a su versatilidad y a la gran cantidad de componentes que ofrece para crear formularios, tablas y botones interactivos, manejo de excepciones y acceso a datos mediante JDBC, lo que garantiza que los sistemas desarrollados bajo estos lineamientos sean robustos, seguros y escalables.

Asimismo, entornos de desarrollo como IntelliJ IDEA han potenciado la productividad de los programadores al brindar editores especializados para construir GUIs con Swing, facilitando el diseño visual y la integración con el código fuente. Esta IDE contribuye a acortar los tiempos de desarrollo y a mejorar la mantenibilidad de los sistemas.

7.2 Funcionalidades implementadas (mínimo 20 RFs):

1. Registro de usuarios.
2. Inicio de sesión con validación de credenciales.
3. Búsqueda de rutas disponibles.
4. Consulta de horarios de salida y llegada.
5. Selección de origen y destino.
6. Visualización de tarifas por ruta.
7. Reserva de pasajes.
8. Emisión de comprobante de compra.
9. Cancelación de reserva.
10. Modificación de datos del pasajero.
11. Listado de todas las reservas activas.
12. Consulta de historial de viajes por usuario.
13. Control de disponibilidad de asientos.

14. Registro de buses en el sistema.
15. Asignación de chofer a cada bus.
16. Reporte de ventas por día.
17. Reporte de rutas más utilizadas.
18. Generación de alertas por falta de disponibilidad.
19. Validación de datos ingresados (nombre).
20. Salida segura del sistema con confirmación.

Manejo de errores y colecciones aplicadas:

- Errores: uso de try-catch para capturar `InputMismatchException` (entradas inválidas), `NullPointerException` (datos faltantes) y validaciones personalizadas (ej. campos vacíos o DNI no válido).
- Colecciones:
 - `ArrayList<Pasajero>` para almacenar clientes registrados.
 - `HashMap<String, Ruta>` para las rutas con clave única.
 - `LinkedList<Reserva>` para gestionar las reservas de forma ordenada.
 - `HashSet<String>` para evitar duplicados en DNIs o correos.

-Ejemplos de código relevantes

```
// Modelo de tabla
modeloTabla = new DefaultTableModel(
    new String[]{"Nombre", "Destino", "Asiento", "Precio"}, 0
);
tablaVentas = new JTable(modeloTabla);
```

Relevancia:

`DefaultTableModel` Agrega registros dinámicamente.

8. PROGRAMACIÓN VISUAL Y ACCESO A DATOS

CODIGO JAVA

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class VentaPasajesGUI extends JFrame {
    private JTextField txtNombre, txtAsiento, txtPrecio;
    private JComboBox<String> cbDestino;
    private JTable tablaVentas;
    private DefaultTableModel modeloTabla;

    public VentaPasajesGUI() {
        setTitle("Registro de Venta de Pasajes TBus");
        setSize(700, 400);
    }
}
```

```

setDefaultCloseOperation(EXIT_ON_CLOSE);
setLocationRelativeTo(null);

// Panel de formulario con márgenes
JPanel panelFormulario = new JPanel(new GridLayout(5, 2, 10, 10));
panelFormulario.setBorder(BorderFactory.createEmptyBorder(15, 15,
15, 15)); // margen interno

panelFormulario.add(new JLabel("Nombre del pasajero:"));
txtNombre = new JTextField();
panelFormulario.add(txtNombre);

panelFormulario.add(new JLabel("Destino:"));
cbDestino = new JComboBox<>(new String[]{"Lima", "Arequipa",
"Cusco", "Puno", "Abancay"});
panelFormulario.add(cbDestino);

panelFormulario.add(new JLabel("Número de asiento:"));
txtAsiento = new JTextField();
panelFormulario.add(txtAsiento);

panelFormulario.add(new JLabel("Precio:"));
txtPrecio = new JTextField();
panelFormulario.add(txtPrecio);

JButton btnRegistrar = new JButton("Registrar Venta");
panelFormulario.add(btnRegistrar);

// Modelo de tabla
modeloTabla = new DefaultTableModel(new String[]{"Nombre",
"Destino", "Asiento", "Precio"}, 0);
tablaVentas = new JTable(modeloTabla);

// Evento botón
btnRegistrar.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        registrarVenta();
    }
});

// Layout principal (márgenes generales)
JPanel contenedorPrincipal = new JPanel(new BorderLayout(10, 10));
contenedorPrincipal.setBorder(BorderFactory.createEmptyBorder(15,
15, 15, 15)); // margen de la ventana

contenedorPrincipal.add(panelFormulario, BorderLayout.NORTH);
contenedorPrincipal.add(new JScrollPane(tablaVentas),
BorderLayout.CENTER);

add(contenedorPrincipal);
}

```



```

private void registrarVenta() {
    String nombre = txtNombre.getText();
    String destino = cbDestino.getSelectedItem().toString();
    String asiento = txtAsiento.getText();
    String precio = txtPrecio.getText();

    if (nombre.isEmpty() || asiento.isEmpty() || precio.isEmpty()) {
        JOptionPane.showMessageDialog(this, "Complete todos los campos",
"Error", JOptionPane.ERROR_MESSAGE);
        return;
    }

    modeloTabla.addRow(new Object[]{nombre, destino, asiento, precio});

    // Limpiar espacios
    txtNombre.setText("");
    txtAsiento.setText("");
    txtPrecio.setText("");
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        new VentaPasajesGUI().setVisible(true);
    });
}
}

```

9. ACTIVIDAD DE RESPONSABILIDAD SOCIAL

Breve descripción de la actividad realizada

Como parte de la responsabilidad social este sistema está orientado a la inclusión tecnológica y la educación digital.

En un taller práctico de capacitación dirigido a personal administrativo y estudiantes de comunidades cercanas, donde se les enseñó el uso de herramientas tecnológicas básicas, el manejo de sistemas informáticos y la importancia de la digitalización en procesos de transporte y servicio al cliente.(presentando el proyecto). En la demostración: se redujo los errores del personal, agilizando la atención al usuario, generando transparencia en los registros del negocio.

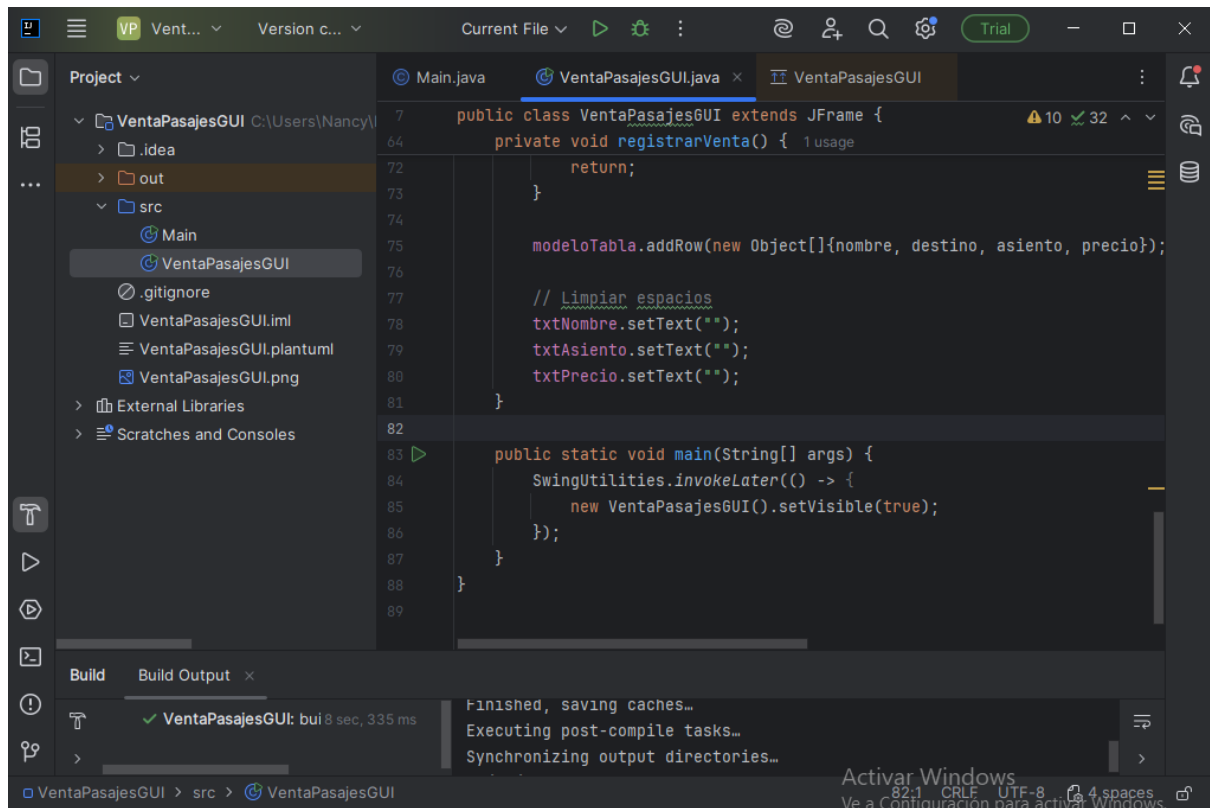
Impacto esperado

- Educativo: Fomentar la aplicación de tecnologías entre personas con conocimientos limitados en informática.
- Social: Promover la inclusión digital, reduciendo la brecha tecnológica en departamentos de provincia.
- Económico: Promueve la formalización del transporte terrestre,brindando mayor confianza de los pasajeros y gestión de ingresos.
- Tecnológico: Incentivar a optimizar procesos en el trabajo a los trabajadores, valorar la importancia de la programación y los sistemas informáticos en la vida real.
- Sostenibilidad: La reutilización del código puede servir como base para que pequeñas empresas de transporte implementen soluciones similares.

10. RESULTADOS

EVIDENCIAS DE EJECUCIÓN DEL SISTEMA Y PRUEBAS REALIZADAS

CAPTURAS DE PANTALLA Y REPORTE



Registro de Venta de Pasajes TBus

Nombre del pasajero:

Destino:

Número de asiento:

Precio:

Nombre	Destino	Asiento	Precio
Manuel Casas	Cusco	15	70
Maria Ravelo	Arequipa	2	90

CONCLUSIONES

Al desarrollar este proyecto consolidamos el aprendizaje en JAVA, utilizando algunos temas presentados en clases del curso, me permitió explorar más funcionalidades de JAVA, explorando IDE'S, fortaleciendo el compromiso personal (like a challenge) y social del proyecto.

Un software puede ser una herramienta de transformación social, económica, genera beneficios más allá del ámbito de negocio, académico.

BIBLIOGRAFÍA

Fuentes Electrónicas y Documentación Técnica:

JetBrains s.r.o. (s.f.). *Design GUI using Swing*. IntelliJ IDEA Help.

<https://www.jetbrains.com/help/idea/design-gui-using-swing.html>

Oracle. (s.f.). *Java Platform, Standard Edition & Java Development Kit Version 25 API Specification*. <https://docs.oracle.com/en/java/javase/25/docs/api/index.html>

Oracle. (s.f.). *Trail: Creating a GUI With Swing*. The Java™ Tutorials.

<https://docs.oracle.com/javase/tutorial/uiswing/index.html>

Tesis y Trabajos Académicos:

Elias Campos, C. P. (2017). *Propuesta de mejora en el área de ventas boletaje nacional, agencia de viajes, AC Tours 2018* (Trabajo de suficiencia profesional)

[Tesis de licenciatura, Universidad San Ignacio de Loyola]. Repositorio USIL.

<https://repositorio.usil.edu.pe/server/api/core/bitstreams/037b468c-f736-4ec4-ad30-b36e7eeead06/content>

Yanapa Quispe, E. (2022). *Desarrollo de un sistema web para dar soporte en la venta de pasajes en la Empresa de Transportes Santacruz Juliaca 2021* [Tesis de grado, Universidad Andina Néstor Cáceres Velásquez]. Repositorio Institucional UANCV.

<https://repositorio.uancv.edu.pe/handle/UANCV/2047>